Генерация кода: Демонстрация

Владимир Языков
Предприниматель, программист, преподаватель

Владимир Языков

О спикере:

- Предприниматель, программист, преподаватель

Сегодня вы узнаете

- Как формулировать структурированные промпты, чтобы генерировать код, готовый к бесшовной интеграции в ваш проект
- Как переводить код с одного языка программирования на другой, сохраняя логику и адаптируя её под идиомы целевого языка
- Как диагностировать корневые причины ошибок и точечно их исправлять, не нарушая работу смежного кода
- Как проводить рефакторинг с помощью ИИ, улучшая читаемость и структуру кода, но строго сохраняя его поведение
- И главное — как на всех этапах оставаться ведущим инженером: ставить задачу, контролировать изменения и верифицировать результат

План занятия

1. Генерация кода с ИИ
2. Транскодинг с ИИ
3. Фиксы с ИИ
4. Рефакторинг с ИИ
5. Заключение

Генерация кода с ИИ

Кейс

Ситуация:
Вы разрабатываете frontend для приложения «TaskManager» на React. У вас уже настроен проект с Vite, подключены основные зависимости (React Query, React Hook Form, Zod, Tailwind CSS). Backend API уже готов и задокументирован.

Задача:
Вам нужно создать модуль для регистрации пользователей. В проекте принят компонентный подход с разделением на слои: components → hooks → api. Все новые файлы должны следовать этому стилю.

Что нужно сгенерировать:

1. API-функцию registerUser для отправки запроса на backend
2. Кастомный хук useRegister с использованием React Query
3. Схему валидации registrationSchema на Zod
4. React-компонент RegistrationForm с формой регистрации

Критически важно:
Код должен быть готов к вставке в существующий проект, использовать принятые в команде библиотеки (React Query, React Hook Form, Zod) и подходы (TypeScript, функциональные компоненты, кастомные хуки)

Промпт

1. Контекст:
Мы пишем frontend для TaskManager. Используем React 19, TypeScript, Vite.

- **Стек:** React Query для серверного состояния, React Hook Form для форм, Zod для валидации, Tailwind CSS для стилей
- **Стиль:** Функциональные компоненты с хуками. API-вызовы вынесены в отдельный слой src/api/. Кастомные хуки в src/hooks/. Компоненты в src/components/
- **API:** Backend доступен по адресу /api. Эндпоинт регистрации: POST /api/auth/register принимает { email: string, password: string } и возвращает { id: number, email: string, createdAt: string }
- **Обработка ошибок:** В проекте есть глобальный обработчик ошибок через React Query onError. При конфликте email (409) нужно показать ошибку в форме

Промпт

2. Задача: Сгенерировать полный набор файлов для функциональности регистрации пользователя:

- **API-функция:** registerUser в файле src/api/auth.ts. Использует fetch или axios (предпочтительно fetch) Типизирована с помощью TypeScript
- **Типы:** UserRepository
- **DTO:**
  - RegisterRequest с полями email (string) и password (string)
  - UserResponse с полями id (number), email (string), createdAt (string)
  - разместить в src/types/auth.ts
- **Схема валидации:** registrationSchema в файле src/schemas/auth.ts. Использует Zod. Email должен быть валидным, пароль минимум 8 символов
- **Кастомный хук:** useRegister в файле src/hooks/useRegister.ts. Использует useMutation из React Query. Возвращает mutate, isPending, error
- **Компонент:** RegistrationForm в файле src/components/RegistrationForm.tsx. Использует useForm из React Hook Form с резолвером Zod. Поля: email, password. Кнопка «Зарегистрироваться». Показывает ошибки валидации и серверные ошибки. При успехе вызывает onSuccess callback из props

Промпт

3. Шаблон ответа:
Предоставь полный код каждого файла, как если бы я мог просто скопировать его в соответствующие директории проекта. Импорты должны быть включены.

4. Критерии качества:

- строго следуй указанным путям и именам файлов
- убедись, что password нигде не логируется
- код должен компилироваться без ошибок TypeScript

DEMO

На что обратить внимание после генерации

1. **Проверка на «галлюцинации»:**
   - **Импорты:** Существуют ли все библиотеки, которые импортировал ИИ? (Например, он мог использовать устаревший API React Query v3 вместо v5)
   - **Зависимости:** Не добавил ли ИИ в package.json (если бы мы его дали) несуществующую или лишнюю библиотеку?

2. **Аудит безопасности:**
   - **Пароль** Убедиться, что поле password не попадает в console.log, в React DevTools или в сетевые логи (кроме самого запроса)
   - **XSS:** Нет ли использования dangerouslySetInnerHTML для пользовательских данных?

3. **Соблюдение границ:**
   - **Логика:** Не добавил ли ИИ лишнюю валидацию (например, на сложность пароля с спецсимволами), которую мы не просили? Это может быть полезно, но требует согласования
   - **Стили:** Использует ли компонент Tailwind CSS, как указано, или ИИ добавил inline-стили или CSS-модули?

Транскодинг с ИИ

Кейс

Ситуация:
Мы продолжаем разработку «TaskManager». В процессе исследования лучших практик мы нашли полезную функцию на Python для расчёта приоритета задач. Нам нужно перенести её в наш Node.js backend на Express

Кейс

Исходный код на Python:

```
from datetime import date, timedelta

def calculate_priority(due_date: date, importance: int) -> float:
    """
    Рассчитывает приоритет задачи.
    :param due_date: дата дедлаина (date object)
    :param importance: важность (целое число от 1 до 10)
    :return: значение приоритета (чем выше, тем срочнее)
    """
    if not isinstance(due_date, date):
        raise TypeError("due_date must be a date")
    if not isinstance(importance, int) or importance < 1 or importance > 10:
        raise ValueError("importance must be an integer between 1 and 10")

    today = date.today()
    days_left = (due_date - today).days

    # Базовыи приоритет: чем меньше днеи, тем выше приоритет
    if days_left <= 0:
        # Просроченные задачи получают максимальныи приоритет
        base_priority = 100.0
    else:
        base_priority = 100.0 / days_left

    # Модификатор важности: умножаем на importance (1..10)
    priority = base_priority * importance

    # Ограничим значение сверху (чтобы не уходило в бесконечность)
         return min(priority, 1000.0)
```

Кейс

Задача:
Перевести эту функцию на TypeScript/Node.js, чтобы встроить её в наш Express-сервис задач. В проекте используется TypeScript, библиотека date-fns для работы с датами.

Критически важно:

- сохранить точность вычислений и все проверки входных данных
- адаптировать обработку ошибок под JavaScript-стиль (throw Error)
- использовать date-fns для работы с датами вместо нативного Date
- избегать «магических чисел» (100.0, 1000.0) – вынести в константы
- код должен быть идиоматичным для TypeScript

Промпт

1. Контекст перевода:

- **Исходный код:** Python 3.10+, использует встроенные типы date и timedelta. Функция предоставлена выше
- **Целевой стек:** Node.js 20, TypeScript 5, Express. Для работы с датами используется библиотека date-fns
- **Контекст проекта:** Функция будет экспортирована из файла src/utils/priorityCalculator.ts. Она будет использоваться в TaskService

2. Задача:
Максимально точно перевести функцию calculate_priority на TypeScript. Создай модуль priorityCalculator.ts с экспортируемой функцией следующей сигнатуры:

```
export function calculatePriority(dueDate: Date, importance: number): number
```

Промпт

3. Критерии качества и ограничения:

- **Сохранение логики:** Поведение должно быть идентичным для всех допустимых и недопустимых входных данных
- **Идиоматичность TypeScript:**
  - используй new Date() или startOfToday() из date-fns для получения текущей даты
  - для проверок входных данных выбрасывай кастомные ошибки: TypeError для невалидной даты и RangeError для неверного importance (вне диапазона 1-10)
  - вынеси магические числа в константы (например, MAX_PRIORITY = 1000, BASE_FACTOR = 100)
  - для вычисления разницы в днях используй differenceInDays из date-fns
- **Обработка граничных случаев:**
  - при daysLeft = 0 (дедлайн сегодня) – basePriority = 100
  - при daysLeft отрицательном (просрочено) – basePriority = 100
  - при daysLeft положительном не происходит деления на ноль
- **Документирование:**
  - добавь TSDoc к функции, описывающий параметры, возвращаемое значение и исключения
- **Стиль:** Следуй ESLint/Prettier конфигурации проекта. Не используй сторонние библиотеки, кроме date-fns

Промпт

4. Шаблон ответа:
Предоставь полный код модуля priorityCalculator.ts с импортами. Добавь краткое пояснение (2-3 предложения), как была адаптирована логика (особенно обработка дат и исключений)

DEMO

На что обратить внимание после генерации

1. **Различия в обработке дат и времени:**
   - Python date.today() возвращает только дату без времени. В JavaScript new Date() включает время. Нужно использовать startOfToday() или startOfDay() из date-fns для корректного сравнения
   - Разница в днях: Python (due_date - today).days возвращает целое число дней. differenceInDays(dueDate, today) из date-fns делает то же самое, но важно проверить порядок аргументов (первый минус второй)

2. Семантические ловушки:

- В Python функция выбрасывала TypeError для не-date и ValueError для некорректного importance. В JavaScript/TypeScript нет ValueError, поэтому используем RangeError или кастомный Error
- В Python проверка isinstance(importance, int) отсекает float. В JavaScript typeof importance === 'number' пропустит float. Нужно добавить проверку Number.isInteger(importance)

3. Адаптация «магии»:

- ИИ вынес числа в константы, но хорошо бы проверить, правильные ли имена выбраны и экспортируются ли они (могут понадобиться для тестов)

Фиксы с ИИ

Кейс

Ситуация:
Модифицировать хук useTaskPriority так, чтобы он корректно обрабатывал случай, когда dueDate равен null или undefined, не меняя при этом поведение для всех остальных случаев.

Бизнес-требование:
Меняем только код хука useTaskPriority в файле src/hooks/useTaskPriority.ts. Сигнатура хука должна остаться совместимой. Другие файлы не затрагиваем.

Диагноз (что мы уже выяснили):
Мы обнаружили, что в хуке useTaskPriority не обрабатывается случай, когда параметр dueDate равен null или undefined. Это приводит к ошибке Invalid time value при вызове differenceInDays. По бизнес-требованиям, задача без дедлайна должна получать фиксированный минимальный приоритет 0.1

Промпт

1. **Задача:**
   Измени хук useTaskPriority так, чтобы он корректно обрабатывал отсутствующий dueDate в соответствии с бизнес-требованиями

2. Границы изменений:

- не меняй публичный интерфейс хука
- не изменяй поведение для случаев, когда dueDate задан — оно должно остаться идентичным текущей реализации
- не добавляй новой функциональности, кроме обработки null/undefined
- сохрани стиль кода: используй понятные имена, вынеси магическое число 0.1 в константу (например, MIN_PRIORITY)

Промпт

3. Критерии успеха и ожидаемое поведение:

- если dueDate равен null или undefined, хук должен вернуть MIN_PRIORITY = 0.1, не выполняя никаких вычислений
- валидация importance должна выполняться всегда (если передан невалидный — вернуть ошибку или дефолтное значение)
- для всех валидных dueDate результат должен быть точно таким же, как и раньше
- хук не должен выбрасывать исключений при dueDate === null

4. Шаблон ответа

Предоставь только изменённую версию хука (можно без повторения всего файла, но с указанием, что константа добавляется на уровне модуля). Добавь краткий комментарий, объясняющий логику обработки null

Промпт

Исходный код хука (для справки):

```
import { useMemo } from 'react';
import { differenceInDays, startOfToday } from 'date-fns';

const BASE_FACTOR = 100;
const MAX_PRIORITY = 1000;

interface UseTaskPriorityProps {
  dueDate: Date;
  importance: number;
}

export function useTaskPriority({ dueDate, importance }: UseTaskPriorityProps): number {
  return useMemo(() => {
    if (importance < 1 || importance > 10) {
      throw new RangeError('importance must be between 1 and 10');
    }

    const today = startOfToday();
    const daysLeft = differenceInDays(dueDate, today);

    let basePriority: number;
    if (daysLeft <= 0) {
      basePriority = BASE_FACTOR;
    } else {
      basePriority = BASE_FACTOR / daysLeft;
    }

    const priority = basePriority * importance;
    return Math.min(priority, MAX_PRIORITY);
  }, [dueDate, importance]);
}
```

DEMO

На что обратить внимание после генерации

1. **Проверка границ изменений:**
   - Убедиться, что ИИ не изменил ничего за пределами указанного метода. В данном случае он добавил только новую константу и условие. Но если бы он, например, переименовал BASE_FACTOR или изменил тип возвращаемого значения — это было бы нарушением

2. Сохранение семантики для существующих случаев:

- Нужно обязательно протестировать, что для всех допустимых дат результат остался прежним. ИИ мог случайно изменить порядок операций или внести ошибку в вычисления. Хорошо бы иметь набор тестов (можно попросить ИИ их сгенерировать на шаге контроля влияния)

3. Логика проверки важности при null **dueDate:**

- Проверить, что ИИ корректно обновил TypeScript-тип UseTaskPriorityProps, сделав dueDate опциональным (dueDate?: Date | null). Если тип не обновлён, TypeScript не позволит передавать null

Рефакторинг с ИИ

Кейс

Ситуация:
Провести рефакторинг компонента RegistrationForm, улучшив его читаемость и структуру, строго сохраняя текущее внешнее поведение.

Желаемые улучшения:

1. Выделить логику формы в кастомный хук useRegistrationForm
2. Разделить компонент на логические части (форма, поля, кнопка)
3. Упростить обработку ошибок, вынеся её в отдельную функцию
4. Применить принцип «один компонент — одна ответственность»

Кейс

Задача:
Провести рефакторинг метода registerUser, улучшив его читаемость и структуру, строго сохраняя текущее внешнее поведение.

Желаемые улучшения:

- Выделить логику хеширования пароля в отдельный приватный метод (или даже отдельный компонент, но пока ограничимся приватным методом в том же классе)
- Разделить метод на логические блоки, возможно, с комментариями
- Упростить проверки, убрав избыточную проверку на null, если она не нужна (но нужно убедиться, что без неё поведение не изменится — т.е. если вдруг придёт null, то должно быть исключение. Можно оставить или полагаться на то, что userMapper выбросит NPE, но это менее понятно. Лучше оставить, но вынести в отдельный метод валидации)
- Применить принцип «одна операция — один метод»

Промпт

Исходный код (только релевантная часть):

```
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';
import { useMutation } from '@tanstack/react-query';
import { registerUser } from '../api/auth';

const schema = z.object({
  email: z.string().email('Введите корректныи email'),
  password: z.string().min(8, 'Минимум 8 символов'),
});

type FormData = z.infer<typeof schema>;

interface RegistrationFormProps {
  onSuccess: () => void;
}

export function RegistrationForm({ onSuccess }: RegistrationFormProps) {
  const {
    register,
    handleSubmit,
    formState: { errors },
    setError,
  } = useForm<FormData>({
    resolver: zodResolver(schema),
  });

  const mutation = useMutation({
    mutationFn: registerUser,
    onSuccess: () => {
      onSuccess();
    },
    onError: (error: Error) => {
      if (error.message.includes('409')) {
        setError('email', { message: 'Этот email уже зарегистрирован' });
      } else {
        setError('root', { message: 'Произошла ошибка. Попробуите позже.' });
      }
    },
  });

  const onSubmit = (data: FormData) => {
    mutation.mutate(data);
  };
```

Промпт

Исходный код (только релевантная часть):

```
  return (
    <form onSubmit={handleSubmit(onSubmit)} className="space-y-4">
      <div>
        <label htmlFor="email" className="block text-sm font-medium">
          Email
        </label>
        <input
          id="email"
          type="email"
          {...register('email')}
          className="mt-1 block w-full rounded-md border-gray-300 shadow-sm"
        />
        {errors.email && (
          <p className="mt-1 text-sm text-red-600">{errors.email.message}</p>
        )}
      </div>

      <div>
        <label htmlFor="password" className="block text-sm font-medium">
          Пароль
        </label>
        <input
          id="password"
          type="password"
          {...register('password')}
          className="mt-1 block w-full rounded-md border-gray-300 shadow-sm"
        />
        {errors.password && (
          <p className="mt-1 text-sm text-red-600">{errors.password.message}</p>
        )}
      </div>

      {errors.root && (
        <p className="text-sm text-red-600">{errors.root.message}</p>
      )}

      <button
        type="submit"
        disabled={mutation.isPending}
        className="w-full rounded-md bg-blue-600 px-4 py-2 text-white hover:bg-blue-700 disabled:opacity-50"
      >
        {mutation.isPending ? 'Регистрация...' : 'Зарегистрироваться'}
      </button>
    </form>
  );
}
```

Промпт

1. Контекст:
Проект «TaskManager», используется React 19, TypeScript, React Hook Form, Zod, React Query. У нас есть компонент RegistrationForm, код которого представлен ниже. Проект следует принципам разделения ответственности, но этот компонент нарушает их — слишком много логики в одном месте

Промпт

2. Задача: Проведи рефакторинг компонента RegistrationForm в соответствии со следующими критериями

3. Критерии рефакторинга (нефункциональные требования):

- **Читаемость:**
  - вынеси логику формы (useForm + useMutation + обработка ошибок) в кастомный хук useRegistrationForm. Хук должен возвращать всё необходимое для рендеринга формы
  - схему валидации вынеси в отдельный файл src/schemas/registrationSchema.ts
  - используй говорящие имена переменных и функций
- **Структура:**
  - создай переиспользуемый компонент FormField для инпута с лейблом и ошибкой
  - компонент RegistrationForm должен остаться «тупым» (presentational) — только разметка и вызов хука
  - типы вынеси в отдельный файл или оставь рядом с хуком
- **Стиль:**
  - следуй стандартному JS-стилю (отступы, порядок методов)
  - сохрани Tailwind CSS классы без изменений
  - следуй конвенциям React (PascalCase для компонентов, camelCase для хуков)
  - не добавляй новые зависимости

Промпт

4. Шаблон ответа:

- предоставь полную версию всех файлов после рефакторинга
- добавь краткое пояснение (2–3 предложения), как была сохранена семантика исходного кода

DEMO

На что обратить внимание после генерации

1. **Изменение поведения (самый главный риск):**
   - Даже небольшие перестановки могут сломать логику. Например, если порядок проверок изменился, или useMutation переместился в другое место, это может повлиять на жизненный цикл
   - В нашем случае логика должна остаться той же: валидация → мутация → обработка ошибок → callback
   - Что проверить: протестировать форму вручную и убедиться, что ошибки отображаются корректно
2. **Скрытое добавление функциональности:**
   - ИИ мог случайно добавить новую валидацию (например, подтверждение пароля) или изменить поведение при ошибке
   - **Что проверить:** убедиться, что схема валидации идентична, и нет новых полей или проверок
3. **Нарушение архитектурных границ:**
   - Выделение хука — безопасно. Но если бы ИИ создал контекст или HOC вместо хука, это изменило бы архитектуру
   - **Что проверить:** хук должен быть простой функцией, не создающей глобального состояния

Заключение

Сегодня вы узнали

1. Мы можем быстро создавать готовый к интеграции код, но только если в промпте точно описан контекст проекта и архитектурные стандарты, иначе результат потребует серьёзной доработки

2. Мы можем переносить алгоритмы между языками, экономя время на переписывании, но должны тщательно проверять граничные случаи и учитывать различия в идиомах, чтобы не внести скрытые ошибки

3. Мы можем точечно устранять ошибки с минимальными изменениями, но только после точной диагностики корневой причины и при жёстком ограничении области правок, иначе можно замаскировать проблему или сломать смежную логику

4. Мы можем улучшать читаемость и структуру кода, делегируя рутину ИИ, но это требует строгого контроля неизменности поведения и обязательной проверки через тесты

Генерация кода: Демонстрация

Владимир Языков
Предприниматель, программист, преподаватель